

Immanuel Peter

ipeter@uchicago.edu | (479) 257-3842 | [linkedin.com/in/immanuel-peter](https://www.linkedin.com/in/immanuel-peter) | github.com/immanuel-peter | ipeter.dev

EDUCATION

The University of Chicago – BS Computer Science, BA Physics | *expected June 2028*

Courses: Machine Learning, Abstract Linear Algebra, Systems Programming, Theory of Algorithms, Foundations of Distributed Systems

EXPERIENCE

Member of Technical Staff Intern | Tensormesh | March 2026 – Present

- Integrated Phoenix into Tensormesh's observability stack, turning router traffic into OpenInference traces and spans.
- Shipped SDK and CLI surfaces for inspecting traces/spans, including async clients, docs, generated references, and tests.
- Productionized rollout configuration across Terraform, Secret Manager, and Cloud Run
- Contributed vLLM Production Stack fixes for runtime defaults and router volume customization.

Software Engineer Intern | Quantum Rings | June 2025 – August 2025

- Reduced API request latency by designing queue-driven telemetry processing that decoupled heavy operations from the API, enabling horizontal scaling (AWS SQS, TypeORM)
- Built full-stack admin analytics dashboards tracking user growth and execution volume with SQL time-bucket aggregation and time zone safe filtering (Next.js, NestJS, Recharts)
- Delivered 19 PRs, 43 contributions, ~15K LOC added across schema migrations, background workers, and KPI dashboards

Open Source Contributor | Meta | October 2025

- Contributed to Pyrefly, Meta's Python type checker in Rust, refactoring error summarization module structures

PROJECTS

AutoMoE — MoE Self-Driving Model | github.com/immanuel-peter/self-driving-model

- Built a 53M parameter modular self-driving stack in PyTorch with 4 expert networks, top-2 gating, and 10-waypoint trajectory prediction.
- Released ~145K CARLA frames across multimodal and multi-camera datasets, and trained/fine-tuned the pipeline with PyTorch DDP on up to 8 GPUs.

CARLA Autopilot Datasets | hf.co/immanuelpeter/datasets | **15K+ Downloads**

- Released two MIT-licensed CARLA autopilot datasets (~145K frames, 567 GB) on Hugging Face with 2.2K and 6.5K downloads each; flagship split includes synchronized 4-camera RGB, 32-channel LiDAR, semantic segmentation, and 2D bounding boxes across 24 weather/map scenarios
- Designed run-based train/val/test splits to eliminate temporal leakage, supporting perception, sensor fusion, imitation learning, and RL benchmarking under extreme conditions

Hostess — Docker Compose for Production | hostess.sh

- Built a deployment platform where teams define a full polyglot stack in one hostess.yml and ship with one command—Next.js, FastAPI, Postgres, and Redis with automatic service wiring, builds, routing, and TLS
- Shipped Git-native continuous deployment with full-stack per-PR preview environments, branch to environment routing, and commit statuses
- Launched Hostess Studio with framework-aware dashboards: Next.js route introspection, FastAPI OpenAPI playground, in-browser Postgres/Redis admin, and route-level metrics with distributed trace waterfalls
- Designed integrated lifecycle for stateful services—scoped multi-tenant databases, automated provisioning, backup/restore with safety snapshots, and port-forwarding for secure local database access

Redis Kubernetes Operator | github.com/howl-cloud/redis-operator

- Engineered a production-grade Kubernetes operator in Go managing Redis standalone, sentinel, and cluster modes with direct pod orchestration for deterministic failover
- Achieved 606 reconciles/sec by implementing fencing-first failover with split-brain prevention, boot-time role guards, and supervised primary updates
- Built chaos test suite (network partition, primary kill, operator restart), Prometheus metrics, Grafana dashboards, and Helm chart with OCI registry publishing

Matchbox — AI Research Matching

- Built an AI-driven research lab matchmaking platform connecting students with labs using vector search and LLM-based scoring for semantic matching (Next.js, FastAPI, ChromaDB, OpenAI)
- Architected scalable backend infrastructure on GCP using Docker, Terraform, and Cloud Run with CI/CD pipelines for automated deployment and rapid iteration